```
iex> KeywordList.maybe_print(1, print?: false, example: "unused")
1
```

You didn't need to put the keyword list in [] brackets. This is a syntax convenience when a keyword list is the last argument to a function.

Although this syntax seems to be visually similar to keyword arguments in Ruby, the differences between Elixir keyword lists and Ruby keyword arguments are actually quite significant.

The Keyword[2] module has all of the functions you'll need to read or modify keyword lists. You'll commonly use the functions Keyword.get/3, Keyword.put/3, and Keyword.merge/2.

Let's move on to another core data type—maps.

Maps—Your Data Layer Foundation

Maps are the most important data type in your Elixir toolbox. A map consists of key-value pairs, where the key and value can be any data type. In practice, you'll use maps to hold and pass around all of the data in your application.

The Basics

Maps are created with the %{} syntax. You can create an empty map or set the initial key-value pairs in the map. Try it out in IEx:

```
$ iex
iex> empty = %{}
%{}

iex> populated = %{key: "value", another: "entry"}
%{another: "entry", key: "value"}

iex> Map.get(populated, :another)
"entry"

iex> populated[:another]
"entry"

iex> populated.another
"entry"
```

You'll notice that the second map came out in a different order than we typed. Maps aren't in the same order as you added the keys. Smaller maps (of 32 keys or less) are ordered, but you should never rely on the order of a map.

2. https://hexdocs.pm/elixir/1.14/Keyword.html